

High-Fidelity SPAD Image Reconstruction via Cascaded Residual Networks

Project Report — Junwen Gu, University of Wisconsin–Madison (2025)

Abstract

Single-Photon Avalanche Diodes (SPADs) enable extreme low-light imaging by detecting individual photon events, but the resulting measurement data suffers from severe signal-dependent Poisson noise. Traditional image-processing pipelines—designed for additive Gaussian noise—fail under these conditions. This project introduces a Two-Stage Cascaded SE-ResUNet for reconstructing high-resolution (800×800) RGB images from 48-channel SPAD burst sequences.

Stage 1 performs coarse structural reconstruction, while Stage 2 applies global residual refinement to restore textures and suppress photon shot noise. We also develop a high-performance training pipeline optimized for H100 GPUs with improvements in I/O throughput, multiprocessing, VRAM/RAM caching, and asynchronous writeback. Our approach achieves **31.45 dB PSNR** and **0.8807 SSIM**, outperforming baseline methods by more than **+8 dB**.

1 Introduction

SPAD sensors quantize photon arrivals into binary temporal histograms. Although these sensors offer unparalleled low-light sensitivity, the measurements are extremely sparse and dominated by shot noise, whose variance equals its mean (Poisson process). As a result:

- Gaussian-based denoisers (e.g., BM3D) fail.
- Neural networks hallucinate textures under heavy noise.
- Training is bottlenecked by I/O throughput rather than GPU compute.

This project addresses both the algorithmic challenge (reconstructing perceptually realistic images without hallucination) and the system challenge (processing 100GB+ of SPAD data efficiently). We propose a physics-informed cascaded network and an optimized training pipeline.

2 Data Engineering and Preprocessing

2.1 Data Structure and Alignment

Different dataset chunks exhibited inconsistent dimensionality:

- Some chunks: $(1024, H, W)$
- Others: $(1024, H, W, 3)$

We designed a self-adaptive preprocessing pipeline that detects and normalizes shapes, ensuring consistent 48-channel tensors for both training and inference.

2.2 Key Preprocessing Steps

- **Temporal slicing:** retain final 192 bins with the most informative photon events.
- **Unpackbits:** decode binary bitstreams into histogram tensors.
- **Gaussian blur** ($\sigma = 0.6$): suppress extreme shot noise to stabilize CNN input.
- **Log + percentile normalization:** compress dynamic range for stable optimization.

These steps significantly improve early-stage reconstructions and model convergence.

3 Methodology

3.1 Two-Stage Architecture Overview

We adopt a **coarse-to-fine** reconstruction philosophy:

1. **Stage 1: Coarse Reconstruction (SE-ResUNet)** Recovers global structure, geometry, and illumination.
2. **Stage 2: Refinement Network** Learns the global residual and restores textures while suppressing shot noise.

This decomposition allows stable training and prevents hallucinations that appear in single-stage networks.

3.2 Stage 1: SE-ResUNet

Input tensor shape: (1, 48, 800, 800).

Architecture features:

- Residual encoder–decoder with skip connections.
- Squeeze-and-Excitation blocks to recalibrate photon channels.
- Outputs structurally accurate images but with color noise and blotchy textures.

3.3 Stage 2: Global Residual Refinement

Instead of predicting the full RGB image, the Stage 2 network learns:

$$R = I_{GT} - I_{Pred}$$

Final output:

$$I_{Out} = I_{Pred} + R$$

Advantages:

- Faster convergence.
- Sharper textures.
- Fewer hallucination artifacts.

4 Training Strategy

4.1 Loss Functions

We use a composite loss:

- L1 loss (pixel fidelity)
- SSIM loss (local contrast + structure)
- Optional: perceptual loss (VGG features)

4.2 Optimization

- AdamW optimizer
- Cosine annealing learning-rate scheduler
- AMP mixed precision for efficient VRAM usage

4.3 EMA and Curriculum Sampling

EMA (Exponential Moving Average) smooths stochastic gradients and improves generalization, boosting PSNR by ~ 1 dB.

Weighted sampling: high-detail scenes (e.g., Trainset5) are oversampled by $20\times$ in Stage 2 to prevent catastrophic forgetting.

5 Engineering Optimizations

Training is bottlenecked not by GPU compute, but by:

- CPU-bound bit unpacking
- Online decompression latency
- I/O throughput

We introduce several system-level optimizations.

5.1 In-Memory I/O

Using the Linux in-memory filesystem:

`/dev/shm`

instead of SSD improves decompression speed by $10\times$.

5.2 Multiprocessing Pipeline

A redesigned `BatchPredDataset` enables:

- `num_workers = 16`
- No Jupyter deadlocks
- 95%+ sustained GPU utilization

5.3 Asynchronous File Writing

A `ThreadPoolExecutor` prevents on-disk writes from stalling training, improving iteration throughput.

Table 1: Quantitative comparison of different training stages.

Model Stage	PSNR (dB)	SSIM	L1 Loss
Stage 1 (Base)	23.77	0.6867	0.0487
Stage 2 (EMA)	30.56	0.8734	0.0182
Stage 2 (EMA2 Hybrid)	31.45	0.8807	0.0165

6 Experiments and Results

6.1 Quantitative Performance

The cascaded design provides more than an **+8 dB** improvement over baseline reconstructions.

6.2 Qualitative Results

Extremely dark scenes: Stage 1 restores boundaries; Stage 2 removes structured artifacts, improving PSNR from ~ 13.5 dB to ~ 30 dB.

High-frequency textures: EMA2 preserves microtextures without the oversmoothing common in single-stage U-Nets.

7 Deliverables

A complete and portable inference package includes:

- `inference.py`: one-click reconstruction tool
- `model_arch.py`: SE-ResUNet definitions
- `utils.py`: SPAD decoding and preprocessing
- Trained weights: Stage 1 + Stage 2 (EMA2)

Runs on any PyTorch-supported environment.

8 Conclusion

We present a robust SPAD reconstruction system combining a physics-informed neural architecture with high-performance system engineering. Our Two-Stage SE-ResUNet achieves state-of-the-art performance (31.45 dB PSNR) while maintaining excellent texture preservation. Future work includes temporal attention mechanisms and physics-based loss functions.